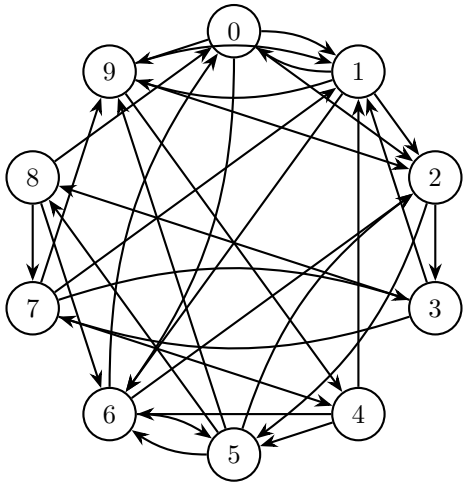
Problem Set 8

Hoa T. Vu

PageRank

Problem 1: PageRank via a Linear System. Consider the directed graph below with 10 nodes (0–9).



The PageRank vector $r \in \mathbb{R}^{10}$ with damping factor $d = 0.85$ satisfies:

$$(I - d\,M)\,r = \frac{1-d}{n}\,\mathbf{1}$$

where M is the column-stochastic transition matrix: $M_{ij} = 1/k$ if there is an edge $j \to i$ (and k is the out-degree of node j).

(a) Build the transition matrix M from the adjacency matrix A below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(Row i = outgoing edges from node i; column j of M sums to 1.)

(b) Set up and solve the linear system $(I - d\,M)\,r = \frac{1-d}{n}\,\mathbf{1}$ using `np.linalg.solve`. Normalise r so it sums to 1.

(c) Which node has the highest PageRank? Does it make sense given the graph structure?

Starter code:

```
import numpy as np

A = np.array([
    [0,1,1,0,0,0,1,0,0,1],
    [1,0,1,0,0,0,1,0,0,1],
    [0,0,0,1,0,1,0,0,0,0],
    [0,1,0,0,0,0,0,1,1,0],
    [0,1,0,0,0,1,1,0,0,0],
    [0,0,1,0,0,0,1,0,1,1],
    [1,0,1,0,0,1,0,0,0,0],
    [0,1,0,1,1,0,0,0,0,1],
    [1,0,0,0,0,0,1,1,0,0],
    [0,1,1,0,1,0,0,0,0,0],
], dtype=float)

d, n = 0.85, 10

# TODO: build M, set up and solve the linear system
```

Problem 2: PageRank via Power Iteration. Using the same graph and damping factor $d = 0.85$, estimate PageRank by repeated matrix multiplication.

Starting from $r^{(0)} = \frac{1}{n}\,\mathbf{1}$, iterate:

$$r^{(t+1)} = d\,M\,r^{(t)} + \frac{1-d}{n}\,\mathbf{1}$$

until $\|r^{(t+1)} - r^{(t)}\|_1 < 10^{-4}$.

(a) Implement the power iteration loop.

(b) How many iterations does it take to converge?

(c) Compare your result with Problem 1 — they should match closely. What is the maximum absolute difference between the two vectors?

Starter code:

```
# M from Problem 1, d = 0.85, n = 10
r = np.ones(n) / n    # uniform start

# TODO: iterate until convergence
```

Least Squares Regression

Problem 3: Linear Regression on Red Wine Quality. Download the dataset from:

`https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009`

The CSV contains 11 physicochemical features and a `quality` score (integer 3–8) for 1 599 red wines.

(a) Load the CSV. Build the design matrix A (11 feature columns plus a bias column of ones) and target vector b (`quality`).

(b) Solve the least-squares problem using `np.linalg.lstsq`. Report the coefficient vector.

(c) Compute the root mean squared error on the training set:

$$\text{RMSE} = \sqrt{\frac{1}{n}\|Ax - b\|^2}.$$

(d) Which feature has the largest absolute coefficient? Does that match your intuition about wine quality?

Problem 4: Quadratic Curve Fitting. The 20 data points below were generated from a hidden quadratic function plus noise.

x	y	x	y
0.00	25.07	4.21	11.47
0.42	14.75	4.63	16.06
0.84	13.11	5.05	22.02
1.26	11.80	5.47	24.36
1.68	6.48	5.89	29.76
2.11	7.81	6.32	32.33
2.53	7.50	6.74	45.12
2.95	2.64	7.16	51.30
3.37	12.07	7.58	59.94
3.79	12.63	8.00	63.42

Fit a quadratic $y = ax^2 + bx + c$ using least squares.

(a) Build the design matrix $A = \begin{bmatrix} x^2 & x & \mathbf{1} \end{bmatrix}$ with three columns.

(b) Solve using `np.linalg.lstsq` and report a, b, c.

(c) Plot the data points and the fitted curve on the same axes.

(d) Compute the RMSE of your fit.

Starter code:

```
import numpy as np

x = np.array([0.00, 0.42, 0.84, 1.26, 1.68, 2.11, 2.53, 2.95,
              3.37, 3.79, 4.21, 4.63, 5.05, 5.47, 5.89, 6.32,
              6.74, 7.16, 7.58, 8.00])
y = np.array([25.07, 14.75, 13.11, 11.80,  6.48,  7.81,  7.50,  2.64,
              12.07, 12.63, 11.47, 16.06, 22.02, 24.36, 29.76, 32.33,
              45.12, 51.30, 59.94, 63.42])

# TODO: build A with columns [x**2, x, ones], then call lstsq
```